Phase 2: Complex Patterns (v2.0)

The modules that make canopy/dnd exceed every existing library. These are the patterns users actually build -- kanban boards, tree editors, multi-select file managers -- but every library makes you assemble from raw primitives. We provide them as first-class APIs.

Depends on: Phase 1 (Animation, Handle, Hitbox)

Tasks

2.1 DnD.Tree -- Tree Drag-and-Drop

Priority: Critical (file explorers, nav menus, org charts -- extremely common)

No DnD library provides a dedicated tree API. Pragmatic DnD has an example, dnd-kit requires hundreds of lines of custom code, react-arborist is React-only and not composable. This is our biggest opportunity to be definitively best-in-class.

Deliverables:

- [ ] `TreeNode id` type -- recursive tree structure with `children` and `isExpanded`
- [ ] `Config id msg` -- tree DnD configuration
- [ ] `TreeNodeState` -- visual state per node (isDragging, isOver, dropPosition, depth, isAncestorOfDragged)
- [ ] `DropPosition` type -- `Before | After | Inside`
- [ ] `view` -- render full tree with DnD enabled
- [ ] `flatten` -- flatten tree to list for collision detection
- [ ] `applyMove` -- apply a tree reorder operation
- [ ] `isValidMove` -- cycle detection (can't drop parent into own child)
- [ ] `expandNode` / `collapseNode` -- tree manipulation during drag
- [ ] Auto-expand on hover (configurable delay, default 500ms)
- [ ] `maxDepth` enforcement -- prevent nesting beyond limit
- [ ] `canDropInto` / `canDropBefore` predicates
- [ ] Indent guides in view
- [ ] Keyboard navigation: arrow keys navigate tree, Space/Enter grab, arrows move position, Tab cycles Before/After/Inside

Implementation notes:

- Flatten the tree to a visible-items list for rendering and collision detection
- Each visible item is a droppable with Hitbox zone detection:
  - Top 25% → `Before` (drop as sibling before)
  - Bottom 25% → `After` (drop as sibling after)
  - Middle 50% → `Inside` (drop as last child)
- Cycle detection: walk from target up to root, if active item is found → invalid
- Auto-expand: track hover time per node. When drag hovers a collapsed node for `expandOnHoverDelay` ms, fire expand. Use a subscription with `Time.every` or `Process.sleep`.
- Dragging a node drags all its descendants (they disappear from the tree during drag)

- Drop animation: spring to final position using DnD.Animation

Edge cases:

- Dropping on the last visible item's "After" zone when it has hidden children (collapsed)
- Dragging the only child out of a parent (parent becomes childless)
- maxDepth: if target is already at max depth, disable "Inside" drop position
- Empty tree: should still render a drop zone

Tests:

- [ ] Flatten produces correct order (DFS pre-order)
- [ ] Flatten respects isExpanded (collapsed children excluded)
- [ ] applyMove before: inserts as sibling before target
- [ ] applyMove after: inserts as sibling after target
- [ ] applyMove inside: appends as last child of target
- [ ] isValidMove rejects dropping into own descendant
- [ ] isValidMove allows dropping into sibling
- [ ] isValidMove allows dropping into unrelated subtree
- [ ] maxDepth prevents dropping inside a node at max depth
- [ ] expandNode expands collapsed node
- [ ] collapseNode collapses expanded node
- [ ] canDropInto predicate is respected
- [ ] canDropBefore predicate is respected
- [ ] Dragging parent removes it and all descendants from visible list
- [ ] Zone detection: top 25% → Before, bottom 25% → After, middle → Inside

Lines: ~450

2.2 DnD.Kanban -- Kanban Board

Priority: Critical (the #1 DnD use case -- Trello, Jira, Notion, Linear)

Every DnD demo shows a kanban board, but every library makes you build it yourself from primitives. Cross-container movement, WIP limits, empty column handling, column reordering -- it's always 200+ lines of glue code. We make it one function call.

Deliverables:

- [ ] `Column columnId cardId` type
- [ ] `Config columnId cardId msg` -- full kanban configuration
- [ ] `CardMove columnId cardId` -- result type describing what moved where
- [ ] `CardState` / `ColumnState` -- visual state during drag
- [ ] `State columnId cardId` -- kanban-specific state (wraps DnD.State)
- [ ] `init` / `view` / `subscriptions`
- [ ] `applyCardMove` -- apply a move to column data
- [ ] `canDrop` predicate -- WIP limits, permission checks
- [ ] `columnsDraggable` flag -- optional column reordering
- [ ] `viewEmptyColumn` -- empty columns are valid drop targets

- [ ] Separate collision detection for cards vs columns
- [ ] Keyboard: Tab between columns, arrows within column, Ctrl+arrow to move across columns
- [ ] Announcements: "Card 'Fix bug' moved from 'In Progress' to 'Done', position 3 of 7"

Implementation notes:

- Internally manages two DnD contexts: one for cards (cross-container sortable) and optionally one for columns (horizontal sortable)
- Cards use vertical Sortable within each column
- Cross-column movement: detect when card hovers a different column's droppable zone, move to that column
- Column drop zones extend to the full column height including empty space below cards
- `canDrop` returning False grays out the column and prevents drop
- Hitbox closestEdge determines insert position within column (before/after existing cards)
- Animation: displaced cards spring to new positions, moved card springs to final position on drop

Edge cases:

- Dragging the last card out of a column (column becomes empty, shows empty state)
- Dropping on empty column (no card targets, but column itself is the target)
- Column reorder while a card is being dragged (should not be possible -- disable column drag during card drag)
- Very tall columns with scroll -- auto-scroll within column during drag
- WIP limit reached: visual indicator on column header

Tests:

- [ ] Card moves within same column (reorder)
- [ ] Card moves to different column
- [ ] Card moves to empty column
- [ ] applyCardMove updates data correctly for within-column
- [ ] applyCardMove updates data correctly for cross-column
- [ ] canDrop false prevents move
- [ ] Column reorder works when enabled
- [ ] Column reorder disabled when card drag is active
- [ ] CardMove contains correct fromColumn, toColumn, fromIndex, toIndex
- [ ] Empty column renders viewEmptyColumn

Lines: ~400

2.3 DnD.MultiSelect -- Multi-Item Drag

Priority: High (file managers, image galleries, email clients)

Select multiple items and drag them as a group. No library does this well -- SortableJS has a basic plugin, everyone else leaves it to the user.

Deliverables:

- [ ] `State id` -- wraps DnD.State + selection set
- [ ] Selection operations: `select`, `deselect`, `toggleSelect`, `selectRange`, `selectAll`, `clearSelection`

- [ ] Selection queries: `selectedItems`, `isSelected`, `selectionCount`
- [ ] `Config id msg` with `onSelectionChange` callback
- [ ] `MultiDragStart` / `MultiDragOver` / `MultiDragEnd` events including selected set
- [ ] `multiDraggable` -- attributes for multi-selectable + draggable items
- [ ] `viewDragCount` -- badge showing "3 items" on drag overlay
- [ ] `reorderMulti` -- reorder preserving relative order of selected items
- [ ] `moveMultiBetween` -- cross-container multi-move

Implementation notes:

- Selection is tracked in a `Set id` inside State
- Click: clear selection, select this item
- Ctrl/Cmd+click: toggle this item in selection
- Shift+click: select range from last-selected to this item (requires knowing item order)
- When drag starts on a selected item: all selected items participate
- When drag starts on an unselected item: clear selection, select+drag only this item
- During drag: selected items "collapse" out of the list, non-selected items fill gaps
- On drop: all selected items insert at target position, preserving their original relative order
- Drag overlay: stack effect (slightly offset shadows) + count badge

Edge cases:

- Shift+click range when items span multiple containers
- Selection across containers in a kanban (select cards from different columns)
- Deselecting during drag (not allowed -- selection is locked during drag)
- Very large selections (100+ items) -- performance of set operations

Tests:

- [ ] Click selects single item, clears previous
- [ ] Ctrl+click toggles selection
- [ ] Shift+click selects range
- [ ] selectAll selects everything
- [ ] clearSelection empties set
- [ ] Dragging selected item includes all selected in drag events
- [ ] Dragging unselected item clears selection, drags only that item
- [ ] reorderMulti preserves relative order of selected items
- [ ] reorderMulti places all items at target position
- [ ] moveMultiBetween removes from source, inserts in target
- [ ] selectionCount is accurate

Lines: ~350

2.4 DnD.Monitor -- Global Drag Observers

Priority: Medium (useful for analytics, global UI effects, debug)

Observe drag operations without being tied to a specific element. Like Pragmatic DnD's `monitorForElements`.

Deliverables:

- [ ] `MonitorConfig msg` -- optional callbacks for each lifecycle event
- [ ] `monitor` -- subscription for global drag events
- [ ] `isAnyDragActive` -- is anything being dragged?
- [ ] `dragType` -- category of current drag (from item data)

Implementation notes:

- Monitor is a subscription that taps into the same State as the DnD context
- Multiple monitors can coexist (different parts of the app observing)
- Monitor events fire after droppable events (like Pragmatic DnD's bubbling model)

Tests:

- [ ] Monitor receives DragStart when drag begins
- [ ] Monitor receives DragOver during drag
- [ ] Monitor receives DragEnd on drop
- [ ] Monitor receives DragCancel on cancel
- [ ] isAnyDragActive is True during drag, False otherwise
- [ ] dragType extracts type from item data

Lines: ~150

2.5 DnD.Operation -- Drag Operation Types

Priority: Medium (move/copy/link semantics, modifier key detection)

Different drag semantics: move (relocate), copy (duplicate), link (create reference). The cursor and drop behavior change based on held modifier keys.

Deliverables:

- [ ] `Operation` type: `Move | Copy | Link`
- [ ] `fromModifierKeys` -- determine operation from Ctrl/Alt/Shift state
- [ ] `defaultOperationMap` -- standard mapping (no mod = Move, Ctrl = Copy, Alt = Link)
- [ ] `cursorForOperation` -- CSS cursor string per operation
- [ ] `withOperation` -- attach operation awareness to DnD config

Implementation notes:

- Modifier key state is available from pointer events (`ctrlKey` , `altKey` , `shiftKey`)
- Operation can change mid-drag as user presses/releases modifier keys
- Cursor changes in real-time to indicate current operation
- The `onDragEnd` event includes the operation so the handler knows whether to move, copy, or link

Tests:

- [ ] No modifiers → Move
- [ ] Ctrl → Copy
- [ ] Alt → Link
- [ ] Cursor strings are valid CSS
- [ ] Operation changes when modifier state changes

Lines: ~120

2.6 DnD.Snap -- Alignment Guides

Priority: High (Figma-style -- no other DnD library has this)

Snap to other elements' edges/centers with visible alignment guide lines. This is our chance to be uniquely best-in-class. Design tools all have this but no DnD library provides it.

Deliverables:

- [ ] `SnapConfig id` -- targets, threshold (default: 5px), showGuides, endOnly
- [ ] `SnapTarget id` : `SnapToElement id | SnapToGrid { x, y } | SnapToCoordinates (List Float) (List Float)`
- [ ] `GuideLine` type with orientation + position
- [ ] `detectSnap` -- during drag, find snap matches and adjust delta
- [ ] `viewGuides` -- render dashed alignment lines spanning the container
- [ ] Support snapping to: left edge, right edge, horizontal center, top edge, bottom edge, vertical center of target elements
- [ ] Multiple guides active simultaneously (aligned left AND top)
- [ ] `endOnly` mode: free movement during drag, snap only on release (interact.js pattern)

Implementation notes:

- Collect all snap targets' edges and centers: `getLineGuideStops()` (Konva.js pattern)
- For dragged element: identify 6 snap edges (left, hCenter, right, top, vCenter, bottom)
- Compare each snap edge against all guide stops. If distance < threshold, snap and show guide.
- Guides are thin dashed lines (`1px dashed #0096FF`) spanning full container width/height
- Performance: only check currently visible targets (skip if thousands)

Tests:

- [ ] Element snaps when within threshold of target edge
- [ ] Element doesn't snap beyond threshold
- [ ] Horizontal and vertical guides render at correct positions
- [ ] Multiple simultaneous guides work
- [ ] endOnly mode: no snapping during drag, only on release
- [ ] Grid snap still works alongside element snap
- [ ] Guide lines are removed on drag end

Lines: ~250

2.7 DnD.Resize -- Resize Handles

Priority: High (needed by Grid, page builders, dashboard tiles)

Resize as a first-class drag operation, not just for Grid but reusable anywhere. Handles on 8 positions (4 corners + 4 edges). Separating this from Grid makes it composable.

Deliverables:

- [ ] `ResizeHandle` type: `N | S | E | W | NE | NW | SE | SW`
- [ ] `ResizeConfig id msg` -- callbacks, handles list, min/max size, aspect ratio lock, snap

- [ ] `ResizeEvent id` -- handle, delta width/height, new rect
- [ ] `resizable` -- returns element attrs + handle attrs
- [ ] Resize from top/left edges adjusts position (opposite edge stays fixed)
- [ ] Aspect ratio lock: maintain width/height ratio during resize
- [ ] Snap: resize snaps to grid or other elements
- [ ] Keyboard: Shift+Arrow to resize from keyboard

Implementation notes:

- Each handle is a small draggable hotspot at the edge/corner
- During resize, track which handle is active and compute delta differently per handle:
  - SE: width += dx, height += dy (simplest)
  - NW: x += dx, y += dy, width -= dx, height -= dy
  - N: y += dy, height -= dy
  - etc.
- Clamp to min/max constraints after each move
- Aspect ratio: compute desired ratio from initial size, constrain one axis to match

Tests:

- [ ] SE resize increases width and height
- [ ] NW resize changes position and decreases size
- [ ] Min size constraint enforced
- [ ] Max size constraint enforced
- [ ] Aspect ratio maintained when locked
- [ ] Resize snaps to grid when configured
- [ ] Each of 8 handles computes correct delta

Lines: ~300

2.8 Tree: Depth-Dependent Drop Targeting

Priority: Medium (unique feature from react-complex-tree)

When dropping at the end of a subtree, horizontal cursor position determines nesting depth. Cursor further right = deeper nesting (child of last item). Cursor further left = shallower (sibling of parent). Brilliant UX, only react-complex-tree has this.

Addition to Task 2.1 (Tree):

- [ ] `depthFromCursor : Bool` in Tree.Config (default: True)
- [ ] `renderDepthOffset : Float` in Tree.Config (default: 24px, same as indentWidth)
- [ ] When dropping after the last visible child of a collapsed parent:
  - Calculate `depth = floor((cursorX - treeLeftEdge) / renderDepthOffset)`
  - Clamp between 0 and (parent depth + 1)
  - Drop as sibling at the calculated depth level
- [ ] Visual indicator shows the depth level with an indent line

Tests:

- [ ] Cursor far right drops as deepest child

- [ ] Cursor far left drops as root-level sibling
- [ ] Depth clamped to valid range
- [ ] Disabled when depthFromCursor = False

2.9 MultiSelect: Lasso/Marquee Selection

Priority: Medium

Draw a selection rectangle to select multiple items at once. Three selection modes.

Addition to Task 2.3 (MultiSelect):

- [ ] `SelectionMode` : `Contained | Intersected | CenterPoint`
- [ ] Shift+drag on empty area starts lasso
- [ ] During lasso: render semi-transparent rectangle, highlight items matching mode
- [ ] On lasso release: add matched items to selection set
- [ ] `viewLasso : State id -> Html msg` -- render the selection rectangle

Tests:

- [ ] Contained: only items fully inside rectangle are selected
- [ ] Intersected: items touching rectangle edge are selected
- [ ] CenterPoint: items whose center is inside are selected
- [ ] Lasso on empty area doesn't start item drag

2.10 Cross-Container Keyboard Navigation

Priority: Medium

Tab between containers while in keyboard drag mode. Essential for kanban boards via keyboard.

Addition to Keyboard Sensor:

- [ ] During active keyboard drag, Tab moves focus to the next container
- [ ] Shift+Tab moves to previous container
- [ ] Dragged item transfers to the focused container
- [ ] Announcements: "Moved to column 'In Progress'"

Prior art: svelte-dnd-action Tab-between-zones

Summary

Task	Lines	Priority	Dependencies
2.1 Tree	450	Critical	Hitbox, Animation, Handle
2.2 Kanban	400	Critical	Sortable, Animation, Hitbox
2.3 MultiSelect	350	High	Draggable, Sortable
2.4 Monitor	150	Medium	Core State

2.5 Operation	120	Medium	Core Config
2.6 Snap/Guides	250	High	Core types, Modifier
2.7 Resize	300	High	Sensor, Modifier
2.8 Tree depth targeting	+50	Medium	Tree (2.1)
2.9 Lasso selection	+80	Medium	MultiSelect (2.3)
2.10 Cross-container kbd	+50	Medium	Sensor
Total	**~2200**		

Implementation Order

1. **Snap** and **Resize** are independent -- can start immediately alongside Tree/Kanban
2. **Tree** and **Kanban** can be developed in parallel (both depend on Phase 1 but not each other)
3. **MultiSelect** can start once Tree or Kanban is done (shared patterns)
4. **Monitor** and **Operation** are independent and can be done anytime
5. **Depth targeting** (2.8) extends Tree, **Lasso** (2.9) extends MultiSelect, **Cross-container kbd** (2.10) extends Sensor -- do after their parent tasks

Done Criteria

- Tree: file explorer example works with expand-on-hover, cycle detection, maxDepth, depth-dependent targeting
- Kanban: Trello-style board works with cross-column move, WIP limits, column reorder
- MultiSelect: file manager example with Ctrl+click, Shift+click, lasso, group drag
- Snap: Figma-style alignment guides appear during drag near other elements
- Resize: dashboard tile can be resized from all 8 handles with aspect ratio lock
- Monitor: debug overlay shows live drag state
- Operation: Ctrl-drag copies item instead of moving
- Cross-container keyboard: Tab transfers items between kanban columns
- All keyboard interactions work with screen reader announcements
- All tests pass, zero warnings